РАБОТА С GIT И PULL REQUEST

ВАЖНО! НЕ ПУШИТЬ ИЗМЕНЕНИЯ НАПРЯМУЮ В develop!
НА КАЖДУЮ НОВУЮ ЗАДАЧУ СОЗДАЕМ НОВУЮ ВЕТКУ

1. ПЕРВОЕ ПОЛУЧЕНИЕ ПРОЕКТА

```
git clone https://github.com/PM-YandexPracticum/SkillSwap_57_5.git
# Скачать проект с GitHub
cd SkillSwap_57_5
# Перейти в папку проекта
```

2. ПЕРЕД НАЧАЛОМ ВЫПОЛНЕНИЯ ЗАДАЧИ

```
git switch develop
# Переключиться на develop
git pull origin develop
# Забрать последние изменения develop с GitHub
git switch -c feat/login-form
# Создать ветку задачи и переключиться на неё
```

3. НАЗВАНИЯ ВЕТОК

Формат

```
тип/краткое-название-задачи
```

Правила

- только английский язык
- только нижний регистр
- слова разделяются дефисами
- название должно объяснять задачу
- если есть номер задачи, указываем его

Типы веток

```
feat/       новая функциональность
fix/        исправление ошибки
refactor/   рефакторинг
style/      изменение стилей
test/       добавление тестов
docs/       документация
```

Примеры

```
feat/login-form
feat/57-user-profile
fix/header-mobile-menu
```

4. ДОБАВЛЕНИЕ ФАЙЛОВ

Сначала проверяем изменённые файлы:

```
git status
```

Добавляем только конкретные файлы, относящиеся к задаче:

```
git add src/components/LoginForm.tsx
git add src/styles/login.css
```

Не используем

```
git add .
git add -A
```

Эти команды могут случайно добавить в коммит посторонние изменения, не относящиеся к задаче.

5. НАЗВАНИЯ КОММИТОВ

Для названий коммитов используем соглашение Conventional Commits.

Формат

```
type(scope): краткое описание
```

Типы

```
feat       новая функциональность
fix        исправление ошибки
refactor   рефакторинг без изменения поведения
style      изменение стилей
test       тесты
docs       документация
```

Примеры

```
git commit -m "feat(auth): add login form"
git commit -m "fix(header): correct mobile menu position"
git commit -m "style(profile): update profile page styles"
```

Правила

- сообщение пишется на английском языке
- описание начинается с маленькой буквы
- точка в конце не ставится
- один коммит содержит одно логическое изменение

6. ПРОВЕРКА ПЕРЕД ОТПРАВКОЙ

```
npm run lint
# Проверить линтер
npm run test
# Запустить тесты
npm run build
# Проверить сборку
git status
# Проверить состояние репозитория
```

Если тестов в проекте ещё нет, npm run test можно пропустить.

7. ОТПРАВКА ВЕТКИ

При первой отправке новой ветки:

```
git push -u origin feat/login-form
# Отправить ветку на GitHub и настроить связь
```

При следующих отправках из этой же ветки:

```
git push
```

8. СОЗДАНИЕ PULL REQUEST

PR направляется из ветки задачи в develop:

```
ветка задачи → develop
```

Например:

```
feat/login-form → develop
```

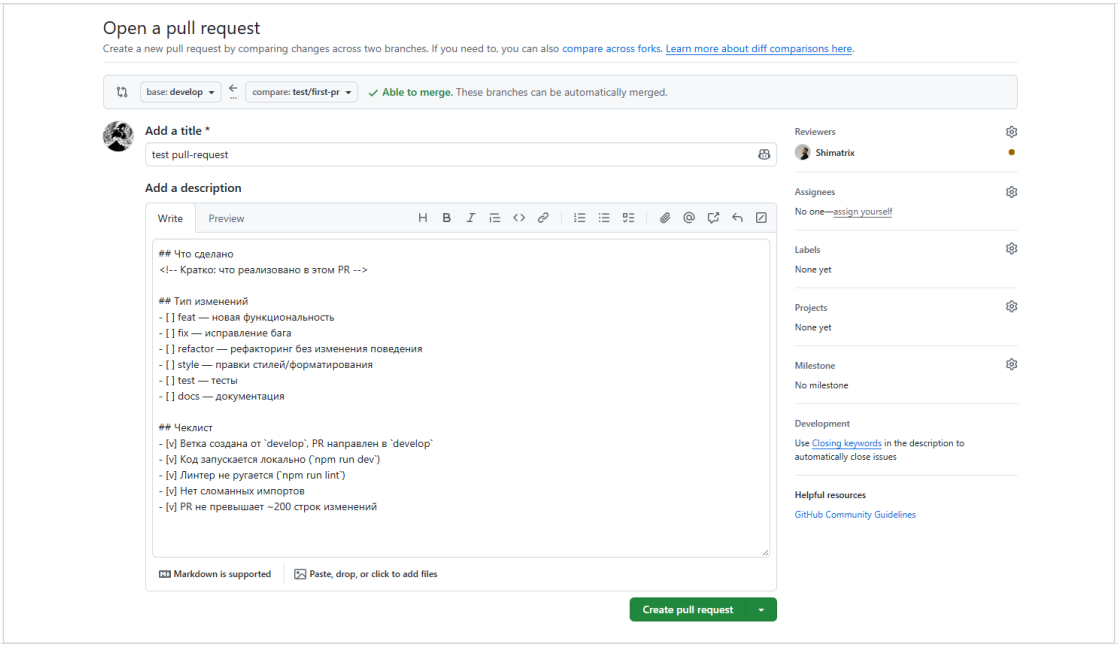
Название PR оформляем по правилам Conventional Commits:

```
feat(auth): add login form
```

В описании PR кратко указываем:

- что сделано
- как проверить изменения
- какие проверки выполнены

И заполняем чеклист.



9. ПРОВЕРКА И СЛИЯНИЕ PR

Автор PR:

- не мержит свой PR самостоятельно
- исправляет замечания в той же ветке
- после исправлений выполняет обычный git push
- не создаёт новый PR для исправлений по этой же задаче

После git push существующий PR обновится автоматически.

10. ПОСЛЕ MERGE

```
git switch develop
# Переключиться на develop
git pull origin develop
# Получить результат merge
git branch -d feat/login-form
# Удалить завершённую локальную ветку
```

11. ПРОВЕРКА ЗАДАЧИ ТИМЛИДОМ ЛОКАЛЬНО

```
git fetch origin
# Получить актуальную информацию и коммиты с GitHub
git branch -r
# Посмотреть удалённые ветки
git switch feature/entities
# Переключиться на ветку задачи и проверить её локально
git pull
# Обновить локальную ветку задачи перед проверкой
```